

Mindanao State University - Iligan Institute of Technology

College of Computer Studies | Department of Computer Applications

BCA182 Embedded Systems Programming

Laboratory Activity No. 1

Real-Time Multisensor Room Monitoring System

Technical Laboratory Report

Student: _____

Subject / Section: _____

Submission date: 29 September 2026

PlatformIO | STM32Cube | STM32F103C8 Blue Pill | FreeRTOS | Wokwi

1. Problem and Requirements

The goal of this laboratory was to build a simulated room monitor on an STM32 Blue Pill. The device is intended to sample temperature, humidity, and relative light; detect motion; show one selected measurement on an OLED; sound an alarm outside the normal temperature range; and enter an inactive state after a period without motion.

The required temperature limits are 18 C and 30 C. The decision logic treats both endpoints as normal: an alarm is low only below 18 C and high only above 30 C. The implementation uses the STM32Cube framework and native FreeRTOS APIs; it does not use the Arduino framework.

Wokwi adapts the physical system into a virtual Blue Pill circuit with a DHT22, photoresistor module, PIR sensor, KY-040 encoder, SSD1306 OLED, and buzzer. The LDR output is expressed as a relative percentage, not calibrated lux.

Implementation boundary: the DHT22 is wired in the Wokwi diagram, but the current sensor code does not communicate with it. It supplies fixed placeholder readings of 25.4 C and 61.2% RH. This report distinguishes those placeholders from real sensor acquisition.

2. System Architecture and Design

Hardware architecture

Device	Pin / interface	Role
STM32F103C8 Blue Pill	MCU; internal HSI clock configured at 8 MHz	Runs application tasks and FreeRTOS
Photoresistor module	PA0, ADC1 channel 0	Relative ambient light input
DHT22	PA1 in the circuit	Intended temperature and humidity input; currently not read by firmware
PIR sensor	PA2	Motion input
KY-040 encoder	PA3 CLK, PA4 DT, PA5 switch	Rotary page selection; firmware samples CLK and DT, but does not read the switch
SSD1306 OLED	PB6 SCL, PB7 SDA, I2C address 0x3C	Displays the selected page
Buzzer	PA8, TIM1 channel 1 PWM	Temperature alarm output
Serial monitor	PA9 TX, PA10 RX, USART1 at 115200 baud	Startup and task diagnostics

The MCU clock setup selects the internal 8 MHz HSI without a PLL. The HAL millisecond tick uses TIM4 because the FreeRTOS port owns SysTick. The OLED transport is a project-local bit-banged I2C implementation on PB6/PB7. The buzzer is configured for a nominal 1 kHz PWM signal.

Software architecture

The application is split across sensor, display, input, alarm, motion, system-state, and RTOS-object modules. The pure decision functions for temperature alarms, encoder navigation, and system-state transitions are separated from peripheral access so they can be compiled for native tests.

```

SensorTask ----> display sensor queue --> DisplayTask --> SSD1306
      +--> alarm sensor queue  --> AlarmTask  --> buzzer
InputTask  -----> display-mode queue  --> DisplayTask
MotionTask -----> EVENT_MOTION  -----> StateTask
StateTask  -----> EVENT_ACTIVE  -----> other tasks
All task diagnostics -----> USART1, guarded by serialMutex

```

DisplayTask is the sole runtime owner of the OLED. SensorTask publishes each sensor record to separate display and alarm queues because a queue receive consumes the item. The event group represents ACTIVE, MOTION, and ALARM status. The serial mutex protects the shared USART1 transmit path.

State machine

```
Boot -> ACTIVE
ACTIVE -- 15 seconds without motion --> INACTIVE
INACTIVE -- motion detected -----> ACTIVE
```

The system starts ACTIVE. StateTask owns EVENT_ACTIVE and applies the 15-second inactivity rule. MotionTask continues sampling the PIR while inactive so activity can wake the system. SensorTask skips acquisition while inactive; InputTask continues polling but suppresses mode changes; DisplayTask blanks the display when it next checks the state; AlarmTask suppresses the buzzer while inactive.

3. FreeRTOS Architecture

All six application tasks are created in main.cpp with explicit priorities. Periodic input, motion, and sensor work uses vTaskDelayUntil() so execution remains aligned to a fixed period instead of accumulating a delay after each task body. Queue and event-group waits place tasks in the Blocked state until their data, event, or timeout is ready.

Task	Responsibility	Trigger / period	Priority	IPC	Typical blocked condition
SensorTask	Read LDR and current temperature/humidity values; publish SensorData to two consumer queues	2 s, vTaskDelayUntil()	2	Two sensor queues; reads EVENT_ACTIVE and EVENT_MOTION	Periodic delay; queue send can wait up to 50 ms
DisplayTask	Render the selected measurement; sole OLED owner	Sensor update or queue timeout	1	Sensor queue and mode queue	Waits up to 2.5 s for a sensor queue item
InputTask	Decode encoder CLK/DT and select a display page while ACTIVE	5 ms, vTaskDelayUntil()	3	Display-mode queue; reads EVENT_ACTIVE	5 ms periodic delay
MotionTask	Sample the PIR and mirror its level to EVENT_MOTION	100 ms, vTaskDelayUntil()	3	System event group	100 ms periodic delay

AlarmTask	Evaluate temperature, drive buzzer PWM, publish EVENT_ALARM	Sensor data; 500 ms queue timeout	2	Alarm sensor queue and system event group	Waits up to 500 ms for sensor data
StateTask	Apply ACTIVE/INACTIVE transitions and inactivity timeout	Motion event or 15 s timeout	2	Waits on EVENT_MOTION; owns EVENT_ACTIVE	Event wait; 500 ms delay after detecting asserted motion

The priorities reflect latency requirements. Encoder and PIR inputs use priority 3 because their signals can change on a short timescale. Sensor, alarm, and state processing use priority 2. OLED updates use priority 1 because small display delays are acceptable and a slow display operation should not outrank input or sampling work.

In FreeRTOS, a task that is eligible to run but waiting for the CPU is Ready; a task waiting for a delay, queue item, or event is Blocked. The periodic tasks return to Blocked after finite work. The implementation uses dynamically created tasks and RTOS objects from the configured FreeRTOS heap.

4. Implementation

SensorData contains temperature, humidity, lightLevel, and motionDetected fields. The current DHT22 function assigns constants; the LDR is sampled through ADC1 and scaled to an integer from 0 to 100. This scale is relative brightness and must not be presented as lux.

DisplayMode is an enumeration with TEMPERATURE, HUMIDITY, LIGHT, and MOTION values. InputTask decodes a falling edge on encoder CLK and compares DT to choose forward or reverse navigation. It posts the new mode to a queue; DisplayTask performs the OLED update. The encoder switch input is configured on PA5 but is not currently processed.

Temperature evaluation is hardware-independent. Values less than 18 C map to LOW_TEMPERATURE, values greater than 30 C map to HIGH_TEMPERATURE, and all values from 18 C through 30 C map to NORMAL. AlarmTask drives TIM1 channel 1 PWM on PA8 when the system is ACTIVE and the temperature is out of range.

The UART mutex is taken in the _write() retarget around each HAL_UART_Transmit() call once the scheduler has started. Startup output is emitted before scheduler operation. The mutex protects access to the peripheral transmit path; it does not wrap an entire printf call as one indivisible record.

5. Verification and Testing

Build and unit tests

Verification	Recorded result	Scope / caveat
--------------	-----------------	----------------

pio run	PASS; last recorded build used 50.2% RAM and 50.4% flash	Firmware compiles and links for bluepill_f103c8
pio test -e native	13 tests passed	Five alarm, four navigation, and four state-logic cases; not peripheral or task integration tests
pio check -e bluepill_f103c8	PASS; 0 high, 0 medium, 10 low findings	Remaining low findings are documented in docs/static-analysis.md
pio check -e native	PASS; no defects reported	Checks only hardware-independent *_logic.cpp files

Wokwi observations

The user-supplied Wokwi screenshot shows the OLED on the HUMIDITY page and a live serial terminal with sample lines showing 25.40 C, 61.25% humidity, 15% light, and no motion. This is useful evidence that a simulation was running, but it does not establish all FT-01 through FT-10 tests.

Evidence discrepancy: the current source assigns humidity 61.2f and formats serial output to two decimal places, which should display 61.20%. The supplied screenshot shows 61.25%. The screenshot may come from another or earlier firmware state. It is therefore recorded as an observed screenshot, not as a verified output of the current checked source.

Test	Required stimulus	Current record
FT-01 / FT-02	Change DHT22 temperature and humidity; confirm the OLED and terminal update	Not verified against current source; DHT22 values are constants
FT-03	Move the LDR from dark to bright and confirm percentage direction	Screenshot shows 15%; dark/bright comparison not recorded
FT-04 to FT-06	Trigger PIR and navigate OLED pages in both directions	Not recorded as a complete test sequence
FT-07	Exercise both alarm limits and normal return	Logic boundaries covered in unit tests; interactive buzzer verification pending
FT-08 to FT-10	Observe inactivity, 15 s timeout, and motion wake-up	Interactive observations pending

No Wokwi run has been independently reproduced for this report. The repository checklist in docs/functional-verification.md leaves these runtime tests pending until each stimulus is performed and its actual result recorded.

6. Static Code Analysis

PlatformIO cppcheck was run for the firmware and native environments. The firmware environment reported no high- or medium-severity findings and ten low-severity findings. The native logic-only analysis reported no defects. The firmware findings are stylistic and related to STM32 CMSIS/HAL register macros and the required HAL callback signature; details and the accepted resolutions are recorded in docs/static-analysis.md.

The native check intentionally excludes STM32-only sources such as the custom port. Its clean result therefore applies to the hardware-independent decision logic, not the complete firmware source set. PlatformIO also emitted nonfatal toolchain-define extraction warnings during the native check.

7. Engineering Discussion

The main architectural trade-off is separating sensor fan-out into two queues. This avoids a race in which DisplayTask and AlarmTask compete to consume the same queue item, at the cost of publishing the same record twice. The event group is used for shared state flags because several tasks need to read current ACTIVE and MOTION levels. The UART uses a mutex because several task contexts emit diagnostics.

The display task waits up to 2.5 seconds on its sensor queue before checking system activity and redrawing. As a result, a mode change or inactive transition may not be reflected immediately. InputTask continues polling while inactive but discards mode changes. The PA5 encoder switch is not consumed. These behaviors are known limitations, not verified feature claims.

The project uses a custom FreeRTOS Cortex-M3 port selected for Wokwi simulator behavior. This is a simulator-specific engineering choice; physical hardware testing has not been documented. The most important current functional limitation is the unimplemented DHT22 protocol, which prevents real temperature/humidity updates and meaningful simulated temperature alarm changes.

8. Conclusion

The project establishes a modular, concurrent STM32 FreeRTOS application with six distinct tasks, queues, an event group, a serial mutex, an OLED display path, PIR-based state logic, and independently testable alarm/navigation/state decisions. The recorded firmware build and 13 native logic tests pass.

The work is not yet a complete functional demonstration of every laboratory requirement. The DHT22 driver is still a placeholder, and the full FT-01 through FT-10 Wokwi verification record remains incomplete. The next engineering steps are to implement real DHT22 acquisition, correct or reconcile the screenshot/source humidity discrepancy, and record each interactive Wokwi test from the current firmware.

References

- BCA182 Embedded Systems Programming, Laboratory Activity No. 1: Real-Time Multisensor Room Monitoring System, September 2026.
- FreeRTOS Kernel API and STM32Cube HAL APIs used by the project.
- Project implementation and verification records in the repository include/, src/, test/, and docs/ directories.

Prepared from the current project source and recorded repository evidence on 29 September 2026. Unperformed tests are identified as pending; no friend-repository test outcomes are claimed as this project's results.